COMS 4130 Project Status Update 1

by Abby Van Der Brink, Taylor Bauer, and Trevor List

COMPLETED

Program Setup (Taylor)

- setup on pyrite; InfraPy repository has been cloned
- Dependency/lib install list: osbpy, scipy, numpy, pyproj, pathos, pisces, ipython, pycallgrah2, coverage
- had to add infrapy to PYTHONPATH permanently
- executed test_beamforming.py with the infrasound sensor array recordings (file YJ.BRP1-4.SAC)

```
python3 test_beamforming.py
Running analysis on time window 600.0 - 610.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 602.5 - 612.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 605.0 - 615.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 607.5 - 617.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 610.0 - 620.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 612.5 - 622.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 615.0 - 625.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 617.5 - 627.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 620.0 - 630.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 622.5 - 632.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 625.0 - 635.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 627.5 - 637.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 630.0 - 640.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 632.5 - 642.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 635.0 - 645.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 637.5 - 647.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 640.0 - 650.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 642.5 - 652.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 645.0 - 655.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 647.5 - 657.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 650.0 - 660.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 652.5 - 662.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 655.0 - 665.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 657.5 - 667.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 660.0 - 670.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 662.5 - 672.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 665.0 - 675.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 667.5 - 677.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 670.0 - 680.0  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 672.5 - 682.5  seconds in frequency band 0.5 - 2.5 Hz...
Running analysis on time window 675.0 - 685.0  seconds in frequency band 0.5 - 2.5 Hz...
```

- found a bug in test_detection.py in the function detect_signals(), it was missing det_p_val as an argument
- fixed it by adding det_p_val=det_thresh in the function call

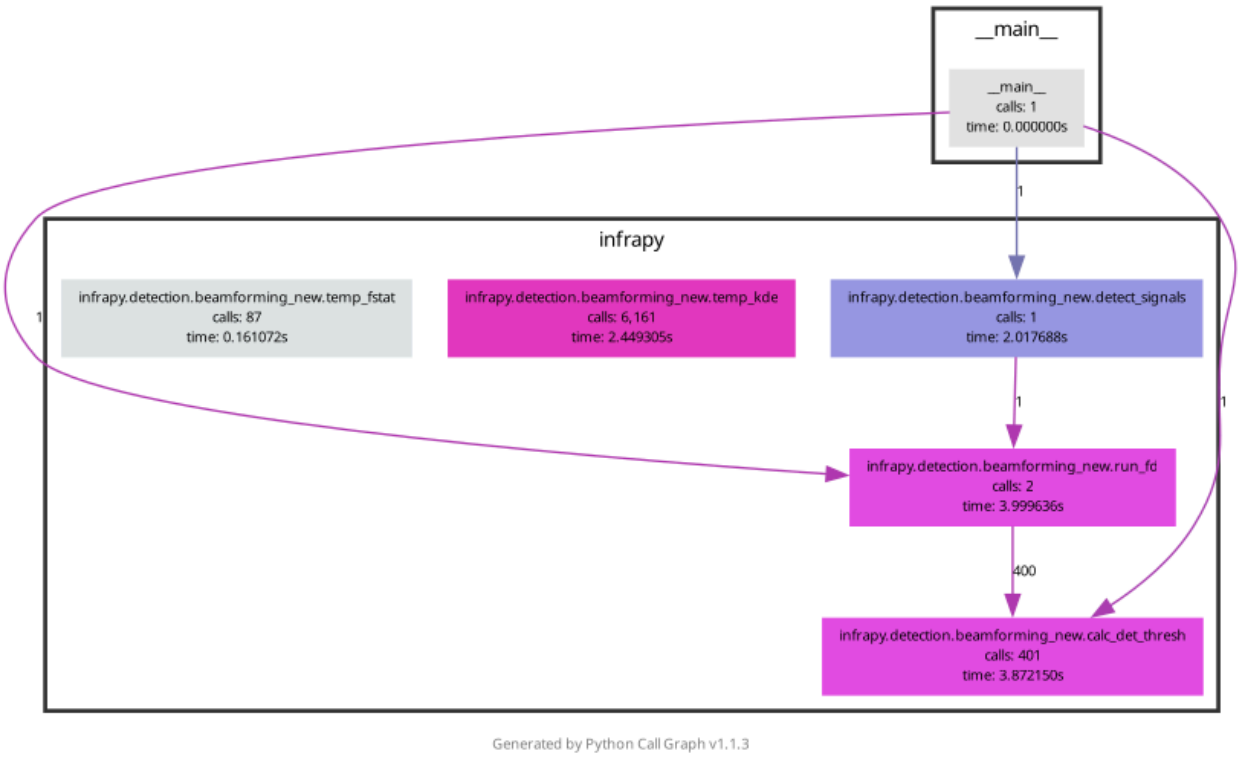
```
tbauer1@pyrite-n3:~/infrapy/examples$ python3 test_detection.py
Traceback (most recent call last):
  File "/home/tbauer1/infrapy/examples/test_detection.py", line 56, in <module>
    dets = beamforming_new.detect_signals(times, beam_results, det_win_len, TB_prod, channel_cnt, min_seq=min_seq, back_az_lim=back_az_lim)
TypeError: detect_signals() missing 1 required positional argument: 'det_p_val'
```

- successfully ran test_detection.py

```
Detection Summary:
Detection time: 2012-04-09T18:11:10.008300      Rel. detection onset: -15.0     Rel. detection end: 107.0       Back azimuth: -112.39   Trace velocity: 337.39   F-stat: 47.33   Array dim: 4
```

Static Analysis (Taylor)

- created script to generate call graph of the fundamental detection decision pipeline with pycallgraph2 & this traces the function call relationships across the functions that are again, foundational to the Adaptive Fisher Detector where the actual detection logic is going on



Some Future Work/Next Steps For Final Project

Static Analysis (this weekend, Taylor)

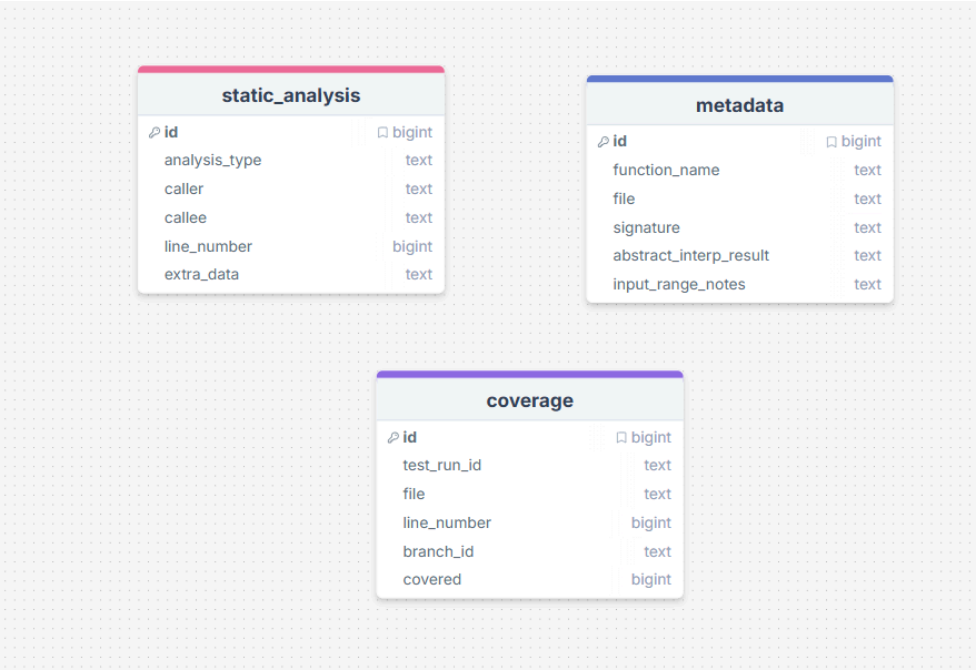
- create beamforming call graph for signal processing pipeline on the sensor files
- full pipeline call graph (running program start to finish on data for final analysis)

Data Flow Analysis (this weekend, Taylor)

Database Setup (within the next week, Taylor)

- Schema and tables that will be created:

- **static_analysis:** Static Analysis Table - will hold call graph edges (caller -> callee relationships), control graph nodes/edges per function, and dataflow dependency pairs
    - id - PRIMARY KEY
    - analysis_type – ‘call_graph’ | ‘cfg’ | ‘dataflow’
    - caller — source function/node
    - callee — target function/node
    - file — source file name
    - line_number — line in source file
    - extra_data — JSON blob for CFG/dataflow detail
- **coverage:** Coverage Table - will map each source line and branch to a boolean cover/uncovered status per test run
    - id - PRIMARY KEY
    - test_run_id — unique ID per fuzzing/test run
    - file — source file name
    - line_number — line in source file
    - branch_id — branch identified
    - covered — 0 = not covered, 1 = covered
- **metadata:** Metadata Table - will store function signatures, the locations of files, and the results of the abstract interpretation
    - id - PRIMARY KEY
    - function_name
    - file — source file name
    - signature — full function signature string
    - abstract_interp_result
    - input_range_notes — documented valid input ranges



Schema made by Abby VDB

Query System Design (Abby):

- Although nothing is yet set up, here is my current design for the query system and how it will work with the code we are analyzing.

Query Type	Retrieval Strategy	Example Question
Call Reachability	Graph Traversal (BFS/DFS on static_analysis)	*"Is function A a (direct/indirect) caller of function B?"*
Coverage Lookup	SQL query on coverage table	*What is the branch coverage of function A under fuzzing test suite?"*
Dataflow Dependency	SQL query on static_analysis (type = dataflow)	*"Are variables A and B data-dependent?"*
Natural Language	Keyword routing to correct template	"Show all functions called during detection"

- **Call Graph Traversal (BFS/DFS):**
    - To answer reachability questions, call graph edges in the static_analysis table will be loaded into an adjacency list and traversed using BFS or DFS

```
import sqlite3, collections
def is_caller(db, func_a, func_b):
   conn = sqlite3.connect(db)
   rows = conn.execute("SELECT caller, callee FROM static_analysis WHERE
analysis_type='call_graph'").fetchall()
   graph = collections.defaultdict(list)
   for caller, callee in rows:
      graph[caller].append(callee)
   #BFS from func_a
   visited, queue = set(), [func_a]
   while queue:
       node = queue.pop(0)
       if node == func_b: return True
       if node not in visited:
         visited.add(node)
         queue.extend(graph[node])
    return False
```

- **SQL Coverage Query:**
    - Branch coverage queries are answered with parameterized SQL:
      SELECT branch_id, covered
      FROM coverage
      WHERE file = 'beamforming_new.py'
      AND function_name = 'run_fk_analysis'
      AND test_run_id = 'fuzz_fun_001';

Abstract Interpretation & Fuzzing (this weekend, Trevor)

- Identify key input variables for the Adaptive Fisher Detector
- Begin defining input range assumptions and abstract interpretation rules for detector behavior
- Review how variables propagate through the detection pipeline to support dataflow analysis

Abstract Interpretation & Fuzzing (next week, Trevor)

- Refine abstract interpretation rules to capture edge cases and potential incorrect detector decisions
- Set up a fuzzing environment using sample soundwave data
- Begin testing detector behavior across varied inputs and document observed anomalies, format fuzzing, and abstract interpretation outputs for database ingestion